

the *i2c_client* names. If any of the names match, then the probe routine of the driver will be called and the *struct i2c_client* will be passed as the parameter to the probe routine. During the probe routine, it is verified that the device represented by the *i2c_client* passed to the driver is the actual device that the driver supports. This is done by trying to communicate with the device represented by *i2c_client* using the address present in the *i2c_client* structure. If this fails, it returns an error from the probe routine informing the Linux device driver subsystem that the device and driver are not compatible; or else it continues with creating device files, registering interrupts and registering with the application subsystem.

The probe skeleton for our example EEPROM system will be as follows:

```
static int eeprom_probe(struct i2c_client *client, const
struct i2c_device_id *id)
{
    check if device exists;
    if device error
    {
        return error;
    }
    else
    {
        do basic configuration of eeprom using
        client->addr;

        register with eeprom subsystem;

        register the interrupt using client->irq;
    }
    return 0;
}

static int adc_probe(struct i2c_client *client, const struct
i2c_device_id *id)
{
    check if device exists;
    if device error
    {
        return error;
    }
    else
    {
        do basic configuration of adc using
        client->addr;

        register with adc subsystem;
    }
    return 0;
}
```

After the probe routine is called and all the required

configuration is done, the device is active and the user space can read and write the device using system calls. For a very clear understanding of how to write *i2c_driver*, refer to the drivers present in the Linux source code—for example, the RTC driver on *i2c bus*, the *drivers rtc/rtc-ds1307.c* file and other driver files.

For reading and writing data on the I2C bus, use the following API.

Reading bytes from the I2C bus:

```
i2c_smbus_read_byte_data(struct i2c_client *client, u8
command);
```

client: *i2c_client* object received from driver probe routine.
command: the command that is to be transferred on the bus.

Reading words from the I2C bus:

```
i2c_smbus_read_word_data(struct i2c_client *client, u8
command);
```

client: *i2c_client* object received from driver probe routine.
command: the command that is to be transferred on the bus.

Writing bytes on an I2C bus:

```
i2c_smbus_write_byte_data(struct i2c_client *client, u8
command, u8 data);
```

client: *i2c_client* object received from driver probe routine.
command: the command that is to be transferred on the bus.
data: the data that is to be written to the device.

Writing words on an I2C bus:

```
i2c_smbus_write_word_data(struct i2c_client *client, u8
command, u16 data);
```

client: *i2c_client* object received from driver probe routine.
command: the command that is to be transferred on the bus.
data: the data that is to be written to the device

When the read or write command is issued, the request is completed using the adapters algorithm, which has the routines to read and write on the bus. 

References

- [1] <http://br.missinglinkelectronics.com/linux/Documentation/i2c/>
- [2] Video on 'Writing and submitting your first Linux kernel patch', <https://www.youtube.com/watch?v=LLBrBBImJt4>
- [3] 'Writing and submitting your first Linux kernel patch' (text file and presentation), <https://github.com/gregkh/kernel-tutorial>

By: Raghavendra Chandra Ganiga

The author can be contacted at rav23ganiga@gmail.com